

NAME : SHAH FAISAL

ROLL NO : 23I-0058

SECTION : BSE – 4A

OPERATING SYSTEMS
LAB 12

BSSE SPRING 2026

LAB 12

QUESTION01

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 10
#define ITERATIONS 100

// Shared resource
int account_balance = 1000; // Initial balance

// Mutex for synchronization
pthread_mutex_t balance_mutex;

// Function to deposit money
void* deposit(void* arg) {
    int* amount = (int*)arg;
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&balance_mutex);
        int current = account_balance;
        usleep(1); // Simulate some work
        account_balance = current + (*amount);
        printf("Deposited: %d, New Balance: %d\n", *amount, account_balance);
        pthread_mutex_unlock(&balance_mutex);
        usleep(1);
    }
    return NULL;
}

// Function to withdraw money
void* withdraw(void* arg) {
    int* amount = (int*)arg;
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&balance_mutex);
        int current = account_balance;
        usleep(1); // Simulate some work
```

```
if (current >= (*amount)) {
    account_balance = current - (*amount);
    printf("Withdrew: %d, New Balance: %d\n", *amount, account_balance);
} else {
    printf("Insufficient balance to withdraw %d! Current: %d\n", *amount, current);
}
pthread_mutex_unlock(&balance_mutex);
usleep(1);
}
return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int deposit_amount = 50;
    int withdraw_amount = 30;

    // Initialize mutex
    if (pthread_mutex_init(&balance_mutex, NULL) != 0) {
        printf("Mutex initialization failed!\n");
        return 1;
    }

    printf("Initial Account Balance: %d\n", account_balance);
    printf("Starting %d threads (5 deposit, 5 withdraw)...\n\n", NUM_THREADS);

    // Create threads
    for (int i = 0; i < NUM_THREADS/2; i++) {
        pthread_create(&threads[i], NULL, deposit, (void*)&deposit_amount);
    }
    for (int i = NUM_THREADS/2; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, withdraw, (void*)&withdraw_amount);
    }

    // Wait for all threads to complete
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\n=== FINAL ACCOUNT BALANCE: %d ===\n", account_balance);
    printf("Expected balance: %d\n", 1000 + (5 * 100 * 50) - (5 * 100 * 30));
}
```

```
printf("Expected: %d\n", 1000 + 25000 - 15000);
printf("Expected: 11000\n");

// Destroy mutex
pthread_mutex_destroy(&balance_mutex);

return 0;
}
shah@DESKTOP-02Q2MPC:~$ gcc lab12_q1.c -pthread -o lab12_q1
shah@DESKTOP-02Q2MPC:~$ ./lab12_q1
Initial Account Balance: 1000
Starting 10 threads (5 deposit, 5 withdraw)...

Deposited: 50, New Balance: 1050
Deposited: 50, New Balance: 1100
Deposited: 50, New Balance: 1150
Deposited: 50, New Balance: 1200
Deposited: 50, New Balance: 1250
Withdrew: 30, New Balance: 1220
Deposited: 50, New Balance: 1270
Withdrew: 30, New Balance: 1240
Withdrew: 30, New Balance: 1210
Deposited: 50, New Balance: 1260
Withdrew: 30, New Balance: 1230
Withdrew: 30, New Balance: 1200
Deposited: 50, New Balance: 1250
Deposited: 50, New Balance: 1300
Deposited: 50, New Balance: 1350
Withdrew: 30, New Balance: 1320
Deposited: 50, New Balance: 1370
Withdrew: 30, New Balance: 1340
Withdrew: 30, New Balance: 1310
Deposited: 50, New Balance: 1360
Withdrew: 30, New Balance: 1330
Withdrew: 30, New Balance: 1300
```

shah@DESKTOP-02Q2MPC: ~

```
Deposited: 50, New Balance: 10580
Deposited: 50, New Balance: 10630
Withdrew: 30, New Balance: 10600
Deposited: 50, New Balance: 10650
Withdrew: 30, New Balance: 10620
Withdrew: 30, New Balance: 10590
Deposited: 50, New Balance: 10640
Withdrew: 30, New Balance: 10610
Deposited: 50, New Balance: 10660
Deposited: 50, New Balance: 10710
Withdrew: 30, New Balance: 10680
Deposited: 50, New Balance: 10730
Withdrew: 30, New Balance: 10700
Deposited: 50, New Balance: 10750
Deposited: 50, New Balance: 10800
Withdrew: 30, New Balance: 10770
Deposited: 50, New Balance: 10820
Deposited: 50, New Balance: 10870
Withdrew: 30, New Balance: 10840
Withdrew: 30, New Balance: 10810
Withdrew: 30, New Balance: 10780
Deposited: 50, New Balance: 10830
Deposited: 50, New Balance: 10880
Deposited: 50, New Balance: 10930
Withdrew: 30, New Balance: 10900
Deposited: 50, New Balance: 10950
Deposited: 50, New Balance: 11000

=== FINAL ACCOUNT BALANCE: 11000 ===
Expected balance: 11000
```

QUESTION02

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
// Two accounts
```

```
int account1 = 5000;
```

```
int account2 = 5000;
```

```
// Mutexes for each account
```

```
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;
```

```
// Safe transfer function - always lock mutexes in same order
```

```
void transfer_safe(int from, int to, int amount, int* from_acc, int* to_acc,
    pthread_mutex_t* first_mutex, pthread_mutex_t* second_mutex) {
```

```
    // Lock mutexes in consistent order to prevent deadlock
```

```
pthread_mutex_lock(first_mutex);
pthread_mutex_lock(second_mutex);

if (*from_acc >= amount) {
    *from_acc -= amount;
    *to_acc += amount;
    printf("Thread %c: Transferred %d from Account%d to Account%d\n",
        (from == 1) ? 'A' : 'B', amount, from, to);
    printf("Thread %c: Account1: %d, Account2: %d\n",
        (from == 1) ? 'A' : 'B', account1, account2);
} else {
    printf("Thread %c: Insufficient balance in Account%d!\n",
        (from == 1) ? 'A' : 'B', from);
}

pthread_mutex_unlock(second_mutex);
pthread_mutex_unlock(first_mutex);
}

// Function to transfer from account1 to account2 (safe version)
void* transfer_1_to_2_safe(void* arg) {
    int amount = 1000;
    sleep(1);

    printf("Thread A: Attempting safe transfer %d from Account1 to Account2\n", amount);

    // Always lock mutex1 first, then mutex2 (by memory address order)
    if (&mutex1 < &mutex2) {
        transfer_safe(1, 2, amount, &account1, &account2, &mutex1, &mutex2);
    } else {
        transfer_safe(1, 2, amount, &account1, &account2, &mutex2, &mutex1);
    }

    return NULL;
}

// Function to transfer from account2 to account1 (safe version)
void* transfer_2_to_1_safe(void* arg) {
    int amount = 1000;
    sleep(1);
```

```
printf("Thread B: Attempting safe transfer %d from Account2 to Account1\n", amount);

// Always lock mutex1 first, then mutex2
if (&mutex1 < &mutex2) {
    transfer_safe(2, 1, amount, &account2, &account1, &mutex1, &mutex2);
} else {
    transfer_safe(2, 1, amount, &account2, &account1, &mutex2, &mutex1);
}

return NULL;
}

int main() {
    pthread_t threadA, threadB;

    printf("=== DEADLOCK RESOLUTION (Consistent Lock Order) ===\n");
    printf("Initial - Account1: %d, Account2: %d\n\n", account1, account2);

    pthread_create(&threadA, NULL, transfer_1_to_2_safe, NULL);
    pthread_create(&threadB, NULL, transfer_2_to_1_safe, NULL);

    pthread_join(threadA, NULL);
    pthread_join(threadB, NULL);

    printf("\nFinal - Account1: %d, Account2: %d\n", account1, account2);

    return 0;
}
```

```
shah@DESKTOP-02Q2MPC:~$ gcc lab12_q2.c -pthread -o lab12_q2
shah@DESKTOP-02Q2MPC:~$ ./lab12_q2
=== DEADLOCK RESOLUTION (Consistent Lock Order) ===
Initial - Account1: 5000, Account2: 5000

Thread A: Attempting safe transfer 1000 from Account1 to Account2
Thread A: Transferred 1000 from Account1 to Account2
Thread A: Account1: 4000, Account2: 6000
Thread B: Attempting safe transfer 1000 from Account2 to Account1
Thread B: Transferred 1000 from Account2 to Account1
Thread B: Account1: 5000, Account2: 5000

Final - Account1: 5000, Account2: 5000
```

QUESTION03

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <errno.h>
#include <time.h>

#define NUM_THREADS 2
#define TIMEOUT_SEC 1
#define MAX_RETRIES 5

pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;

// Function to try locking mutex with timeout
int try_lock_with_timeout(pthread_mutex_t* mutex, int timeout_seconds) {
    struct timespec timeout;
    clock_gettime(CLOCK_REALTIME, &timeout);
    timeout.tv_sec += timeout_seconds;

    int result = pthread_mutex_timedlock(mutex, &timeout);
    return result; // Returns 0 on success, ETIMEDOUT on timeout
}

// Thread 1 with timeout-based deadlock avoidance
void* thread1_safe(void* arg) {
    int retries = 0;

    while (retries < MAX_RETRIES) {
        printf("Thread 1 (Attempt %d): Trying to lock mutex1...\n", retries + 1);
        pthread_mutex_lock(&mutex1);
        printf("Thread 1: Locked mutex1\n");

        printf("Thread 1: Trying to lock mutex2 (with %d sec timeout)...\n", TIMEOUT_SEC);
        int result = try_lock_with_timeout(&mutex2, TIMEOUT_SEC);

        if (result == 0) {
            // Successfully locked both mutexes
            printf("Thread 1: Locked mutex2 - Executing critical section\n");
            sleep(1);
        }
    }
}
```

```
pthread_mutex_unlock(&mutex2);
pthread_mutex_unlock(&mutex1);
printf("Thread 1: Finished successfully\n");
return NULL;
} else if (result == ETIMEDOUT) {
    // Timeout occurred - release mutex1 and retry
    printf("Thread 1: Timeout waiting for mutex2! Releasing mutex1 and retrying...\n");
    pthread_mutex_unlock(&mutex1);
    retries++;
    sleep(1); // Wait before retrying
} else {
    printf("Thread 1: Error locking mutex2\n");
    pthread_mutex_unlock(&mutex1);
    return NULL;
}
}

printf("Thread 1: Max retries reached, giving up\n");
return NULL;
}

// Thread 2 with timeout-based deadlock avoidance
void* thread2_safe(void* arg) {
    int retries = 0;

    while (retries < MAX_RETRIES) {
        printf("Thread 2 (Attempt %d): Trying to lock mutex2...\n", retries + 1);
        pthread_mutex_lock(&mutex2);
        printf("Thread 2: Locked mutex2\n");

        printf("Thread 2: Trying to lock mutex1 (with %d sec timeout)...\n", TIMEOUT_SEC);
        int result = try_lock_with_timeout(&mutex1, TIMEOUT_SEC);

        if (result == 0) {
            // Successfully locked both mutexes
            printf("Thread 2: Locked mutex1 - Executing critical section\n");
            sleep(1);

            pthread_mutex_unlock(&mutex1);
            pthread_mutex_unlock(&mutex2);
```

```
    printf("Thread 2: Finished successfully\n");
    return NULL;
} else if (result == ETIMEDOUT) {
    // Timeout occurred - release mutex2 and retry
    printf("Thread 2: Timeout waiting for mutex1! Releasing mutex2 and retrying...\n");
    pthread_mutex_unlock(&mutex2);
    retries++;
    sleep(1); // Wait before retrying
} else {
    printf("Thread 2: Error locking mutex1\n");
    pthread_mutex_unlock(&mutex2);
    return NULL;
}
}

printf("Thread 2: Max retries reached, giving up\n");
return NULL;
}

// Deadlock simulation functions
void* thread1_deadlock(void* arg) {
    printf("Thread 1: Attempting to lock mutex1...\n");
    pthread_mutex_lock(&mutex1);
    printf("Thread 1: Locked mutex1\n");

    sleep(2);

    printf("Thread 1: Attempting to lock mutex2...\n");
    pthread_mutex_lock(&mutex2);
    printf("Thread 1: Locked mutex2\n");

    printf("Thread 1: Executing critical section\n");
    sleep(1);

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);

    printf("Thread 1: Finished\n");
    return NULL;
}
```

```
void* thread2_deadlock(void* arg) {
    printf("Thread 2: Attempting to lock mutex2...\n");
    pthread_mutex_lock(&mutex2);
    printf("Thread 2: Locked mutex2\n");

    sleep(2);

    printf("Thread 2: Attempting to lock mutex1...\n");
    pthread_mutex_lock(&mutex1);
    printf("Thread 2: Locked mutex1\n");

    printf("Thread 2: Executing critical section\n");
    sleep(1);

    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);

    printf("Thread 2: Finished\n");
    return NULL;
}

int main(int argc, char* argv[]) {
    pthread_t t1, t2;
    int choice;

    printf("=== DEADLOCK SIMULATION & RESOLUTION ===\n");
    printf("1. Simulate Deadlock\n");
    printf("2. Resolve Deadlock with Timeout\n");
    printf("Enter choice (1 or 2): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("\n=== DEADLOCK SIMULATION ===\n");
        printf("Creating two threads that will deadlock...\n\n");

        pthread_create(&t1, NULL, thread1_deadlock, NULL);
        pthread_create(&t2, NULL, thread2_deadlock, NULL);

        printf("Waiting for threads to complete (this will hang due to deadlock)...\n");
        printf("Press Ctrl+C to terminate\n");
    }
}
```

```
pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("This line will never be reached!\n");
} else {
    printf("\n=== DEADLOCK RESOLUTION WITH TIMEOUT ===\n");
    printf("Creating two threads with timeout-based deadlock avoidance...\n");
    printf("Timeout: %d seconds, Max retries: %d\n\n", TIMEOUT_SEC, MAX_RETRIES);

    pthread_create(&t1, NULL, thread1_safe, NULL);
    pthread_create(&t2, NULL, thread2_safe, NULL);

    printf("Waiting for threads to complete...\n\n");

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n=== PROGRAM COMPLETED SUCCESSFULLY ===\n");
    printf("No deadlock occurred!\n");
}

return 0;
}
```

```
shah@DESKTOP-02Q2MPC:~$ ./lab12_q3
=== DEADLOCK SIMULATION & RESOLUTION ===
1. Simulate Deadlock
2. Resolve Deadlock with Timeout
Enter choice (1 or 2): 1

=== DEADLOCK SIMULATION ===
Creating two threads that will deadlock...

Thread 1: Attempting to lock mutex1...
Thread 1: Locked mutex1
Thread 2: Attempting to lock mutex2...
Thread 2: Locked mutex2
Waiting for threads to complete (this will hang due to deadlock)...
Press Ctrl+C to terminate
Thread 1: Attempting to lock mutex2...
Thread 2: Attempting to lock mutex1...
^C
```

```
shah@DESKTOP-02Q2MPC:~$ ./lab12_q3
=== DEADLOCK SIMULATION & RESOLUTION ===
1. Simulate Deadlock
2. Resolve Deadlock with Timeout
Enter choice (1 or 2): 2

=== DEADLOCK RESOLUTION WITH TIMEOUT ===
Creating two threads with timeout-based deadlock avoidance...
Timeout: 1 seconds, Max retries: 5

Thread 1 (Attempt 1): Trying to lock mutex1...
Thread 1: Locked mutex1
Thread 1: Trying to lock mutex2 (with 1 sec timeout)...
Waiting for threads to complete...

Thread 1: Locked mutex2 - Executing critical section
Thread 2 (Attempt 1): Trying to lock mutex2...
Thread 1: Finished successfully
Thread 2: Locked mutex2
Thread 2: Trying to lock mutex1 (with 1 sec timeout)...
Thread 2: Locked mutex1 - Executing critical section
Thread 2: Finished successfully

=== PROGRAM COMPLETED SUCCESSFULLY ===
No deadlock occurred!
shah@DESKTOP-02Q2MPC:~$
```